

PKCERT AI & Software Development Internship

Task 15: Model Persistence & Mini-Project

Abdullah Amir

AI and Software Development
22 July 2026

This report covers the theory and the results. The full code, outputs and plots live in the notebook `model_persistence.ipynb`. The mini-project dataset is the **UCI Cleveland Heart Disease** dataset (303 patients), not used in any earlier task. A Logistic Regression and a Random Forest are trained and compared, the better one is saved with both **pickle** and **joblib**, and the two formats are checked for correctness, size and speed.

Part C: Dataset Selection, EDA & Preprocessing (Mini-Project)

What the features are. Each row is one patient: 13 clinical features – 6 numeric (`age`, `trestbps`, `chol`, `thalach`, `oldpeak`, `ca`) and 7 categorical (`sex`, `cp`, `fbs`, `restecg`, `exang`, `slope`, `thal`) – and a target of whether angiography showed more than 50% coronary narrowing.

Missing values, checked rather than assumed away. Only `ca` (5 missing) and `thal` (2 missing) have gaps, 2.3% of rows. Dropping those 7 rows shifts the positive rate from 45.5% to 45.9% – negligible – but median/most-frequent imputation is used anyway, since it preserves the full 303 rows and is the more complete demonstration of the preprocessing step the task asks for.

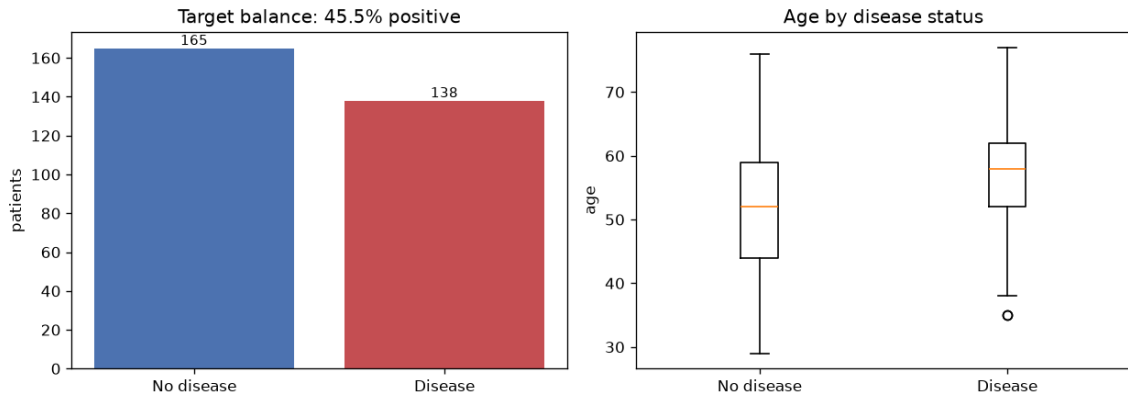


Figure 1: Left: the target is close to balanced, 45.5% positive. Right: patients with disease skew older (median 58 vs 52), as expected, but the overlap is substantial – age alone is a weak signal.

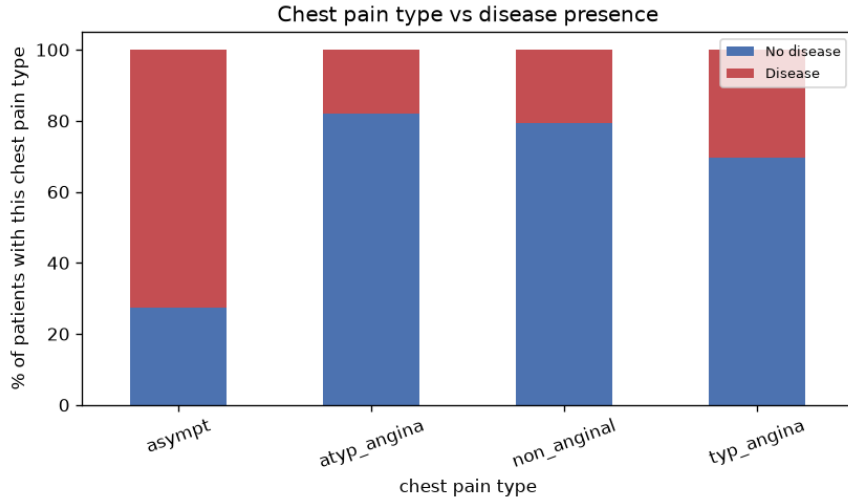


Figure 2: Chest pain type against disease presence.

A counterintuitive detail worth flagging. `asympt` (asymptomatic – no chest pain reported) has the *highest* disease rate of any category, 72.7%, while every named type of pain has a *lower* rate (`typ_angina` 30.4%, `non_anginal` 20.7%, `atyp_angina` 18.0%). The absence of the symptom you would expect to see is the strongest single signal in this feature – exactly the kind of relationship that is easy to miss without checking the crosstab directly.

Split and preprocessing. An 80/20 stratified split (242 train / 61 test rows). A `ColumnTransformer` imputes and standard-scales the numeric columns and imputes and one-hot encodes the categorical ones, wrapped in a single `Pipeline` per model so that preprocessing travels with the model when it is saved.

Model selection

Model	CV ROC-AUC	Test Acc	Precision	Recall	Test ROC-AUC
Logistic Regression	0.9017 ± 0.031	0.8525	0.8519	0.8214	0.9275
Random Forest (300 trees)	0.9037 ± 0.029	0.8033	0.8333	0.7143	0.8837

Table 1: 5-fold cross-validated ROC-AUC on the training set, then held-out test metrics.

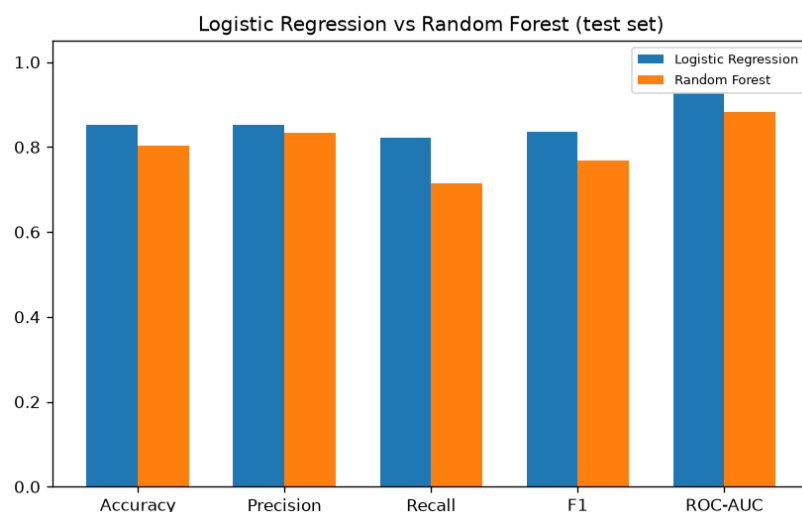


Figure 3: Logistic Regression vs Random Forest on every test-set metric.

The simpler model won, and the two cross-validation scores explain why it is not a fluke. In cross-validation the two models are essentially tied (0.9017 vs 0.9037), but on the held-out test set Logistic Regression pulls ahead on every metric (ROC-AUC 0.9275 vs 0.8837). With only 242 training rows, a 300-tree Random Forest has considerably more capacity to fit sampling noise than a linear model does – a concrete instance of the bias-variance tradeoff rather than a coincidence of one split. Logistic Regression is the model saved and persisted below.

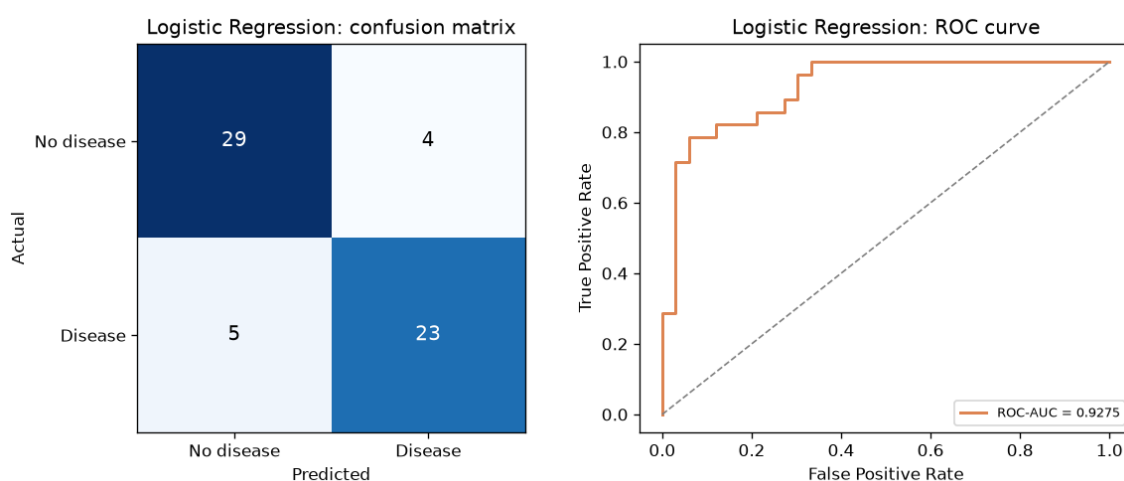


Figure 4: The chosen model's confusion matrix and ROC curve on the test set.

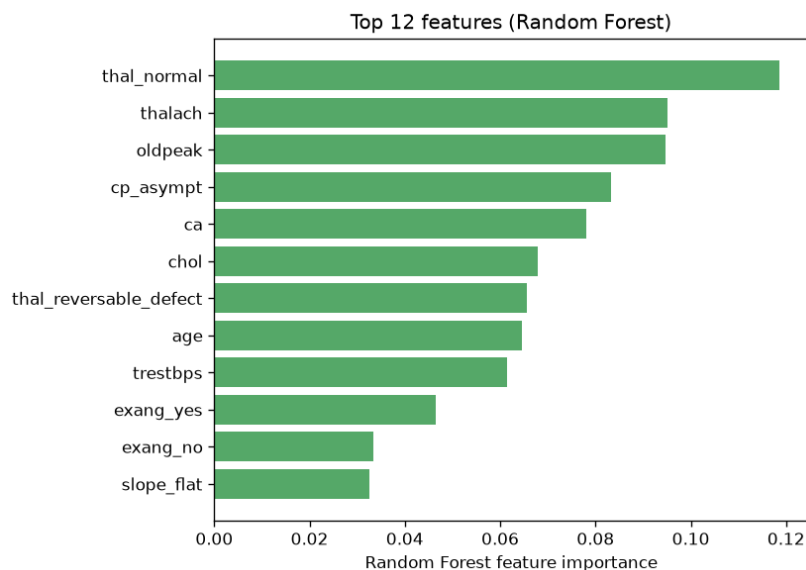


Figure 5: Random Forest feature importance, shown for reference even though Logistic Regression was the model kept. `thal_normal`, `thalach` (max heart rate) and `oldpeak` (ST depression) lead.

Part A: Saving Trained Models

The chosen Logistic Regression pipeline – preprocessing and classifier together – was saved two ways:

Format	File size
pickle (<code>heart_model.pkl</code>)	3,754 bytes
joblib (<code>heart_model.joblib</code>)	5,930 bytes

Part B: Loading & Verifying Models

Both files were loaded back into fresh Python objects and checked against the original in-memory model’s predictions on the test set, both the hard labels and the underlying probabilities.

Both round-trips are bit-for-bit identical to the original: predicted labels match exactly and predicted probabilities match to floating-point precision (`numpy.allclose`), for both the pickle-loaded and the joblib-loaded copy. Serialization lost nothing.

pickle vs joblib, checked rather than recited

The common claim is that joblib is more efficient for scikit-learn models because of how it handles the large numpy arrays inside them. Logistic Regression barely has any (a handful of coefficients), so the same save/load/size/time comparison was repeated on the 300-tree Random Forest – orders of magnitude more numpy array data – to test whether that claim actually holds.

Format	Logistic Regression (small)		Random Forest (300 trees)	
	Size	Load time	Size	Load time
pickle	3,754 B	0.12 ms	2,169,710 B	4.58 ms
joblib	5,930 B	0.63 ms	2,195,451 B	23.21 ms

Table 2: Same comparison, small model and large model.

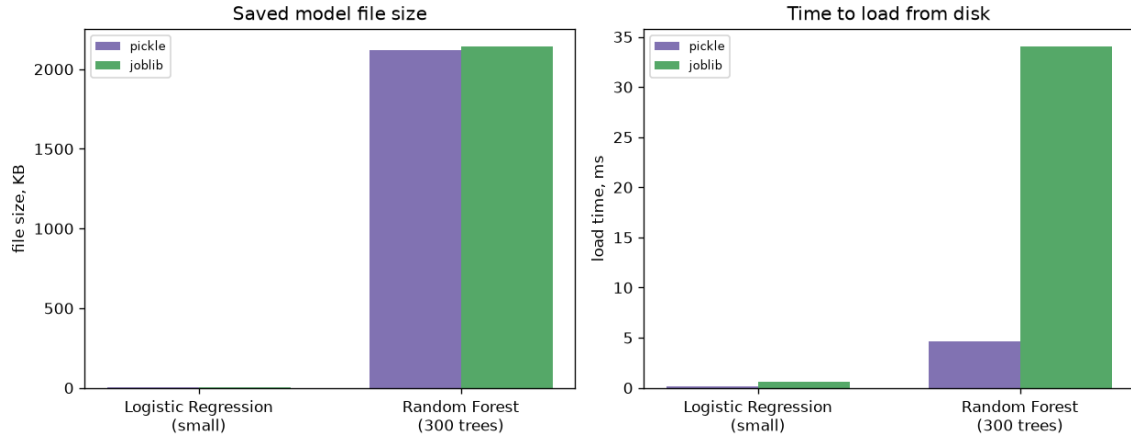


Figure 6: pickle vs joblib, file size and load time, on both models.

The result contradicts the common claim, on both models. pickle was smaller and faster than joblib for the small Logistic Regression *and* for the 2MB Random Forest – joblib was 58% larger for the small model and still 1.2% larger for the large one, and pickle loaded 2–5x faster in both cases.

The likely reason is **PEP 574** (Python 3.8+): pickle protocol 5 added native out-of-band buffer support for numpy arrays, closing most of the efficiency gap joblib was originally built to fix back in 2008, long before protocol 5 existed. On a modern Python interpreter, plain pickle is often just as compact and avoids joblib’s own dispatch overhead.

What joblib still does that pickle does not: a one-argument `compress=` parameter for gzip-style compression without wrapping the file handle manually, and `mmap_mode` for loading a huge array from disk without reading all of it into RAM at once – useful for models far larger than either one tested here. It also remains scikit-learn’s own documented recommendation for persisting estimators, largely for these convenience features rather than raw size or speed.

When to use each. Plain pickle for small models and general Python objects where no extra dependency is wanted; joblib when a model is large enough that `compress=` or `mmap_mode` will matter in practice, or simply to follow scikit-learn’s own convention.

A caveat worth stating plainly. Never unpickle a model file from an untrusted source. Both `pickle.load` and `joblib.load` execute arbitrary code embedded in the file by design – that is what lets them reconstruct arbitrary Python objects – so a malicious `.pkl` or `.joblib` file is a code-execution risk, not merely a data file.

Part D: Model Saving & Documentation

Pipeline summary. Loaded the UCI Cleveland Heart Disease dataset (303 patients, a dataset unused in any earlier task) → checked missing values (2.3% of rows, `ca/thal` only) and confirmed

dropping them barely shifts the class balance → built a `ColumnTransformer` that imputes and scales numeric features and imputes and one-hot encodes categorical ones → trained and cross-validated Logistic Regression and a 300-tree Random Forest → selected Logistic Regression on held-out ROC-AUC (0.9275 vs 0.8837) → saved it with both pickle and joblib → loaded both back and verified bit-for-bit identical predictions → repeated the save/load comparison on the larger Random Forest to test whether joblib’s usual efficiency claim holds.

Key findings.

1. Asymptomatic patients have the *highest* disease rate (73%) of any chest pain category – the absence of a symptom outranks its presence as a signal.
2. The simpler model beat the more flexible one on every held-out metric despite being statistically tied in cross-validation – a concrete instance of the bias-variance tradeoff on a 242-row training set.
3. Both saved-model round-trips reproduced the original predictions exactly, with no loss of precision from serialization.
4. Contrary to the common claim, **pickle beat joblib on both size and speed** for both a tiny model and a 2MB one on this machine – a reminder that “joblib is more efficient” is a claim worth re-checking against the current Python version rather than repeating from memory.

Challenges faced. The dataset’s raw target is a five-level severity scale (0–4) and had to be collapsed to binary (>50% narrowing or not) to match how it is almost universally used in classification benchmarks. The anonymised OpenML mirror exposes columns only as `V1...V13`, so matching them back to their real, documented clinical names required cross-referencing the original UCI column order rather than trusting the mirror alone – the same care that mattered for Day 14’s Bank Marketing dataset. The pickle-vs-joblib size result was the opposite of what is commonly repeated online, which made it tempting to assume a bug in the benchmark; verifying it on a second, much larger model before trusting it was the only way to be sure it was a real result and not noise from one small file.

Conclusion. The thread running through this task is that persistence is not a detail to bolt on after modelling is finished – it is where a model actually becomes useful outside the notebook that trained it, and both format choice and integrity deserve the same scepticism applied everywhere else in this internship. Saving is easy; what is worth checking is whether the thing you load back is truly the thing you saved, and whether the received wisdom about how to save it still holds.

The notebook and README are on GitHub: <https://github.com/AbdullahAmir06/ai-internship-abdullahamir>